Searching in Array

What Is Code Complexity?

- Code Complexity is a measure of how difficult your code is to understand.

Search algorithms

- Search algorithms are a fundamental computer science concept that you should understand as a developer.
- They work by using a step-by-step method to locate specific data among a collection of data.

What is a Search Algorithm?

- According to Wikipedia, a search algorithm is:

Any algorithm which solves the search problem, namely, to retrieve information stored within some data structure, or calculated in the search space of a problem domain, either with discrete or continuous values.

- Search algorithms are designed to check or retrieve an element from any data structure where that element is being stored. They search for a target (key) in the search space.

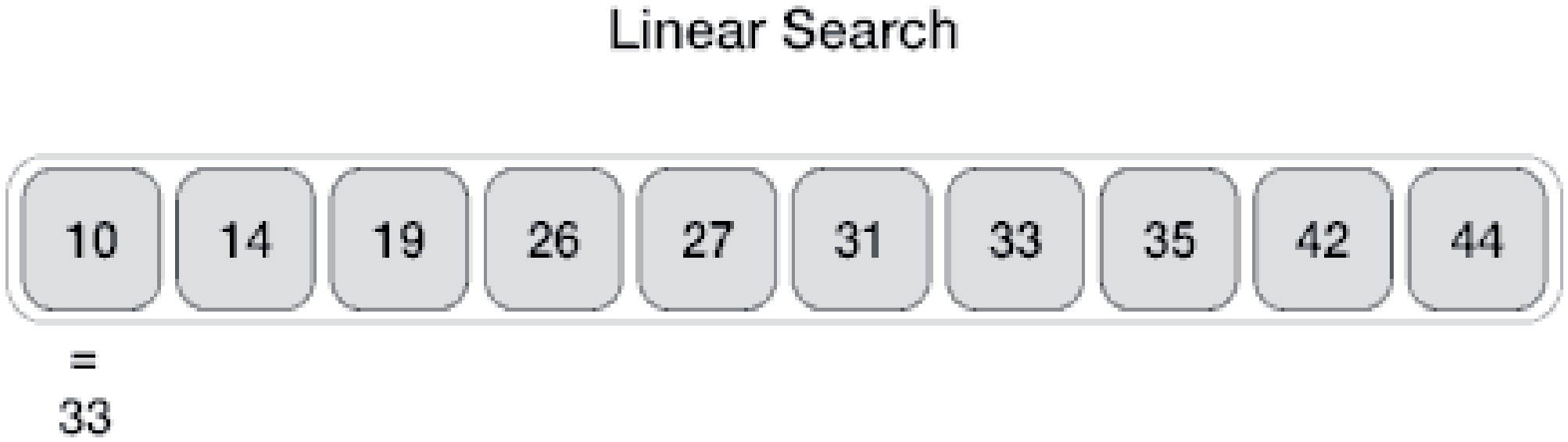
Types of Search Algorithms

1. **Linear or Sequential Search**
2. **Binary Search**

Linear or Sequential Search

- Linear search is the simplest and least performant searching algorithm
- This algorithm works by sequentially iterating through the whole array or list from one end until the target element is found.
- If the element is found, it returns its index, else -1.
- Linear search is not performant because in order to find the element in the worst case the computer will have to examine every single item in the array.
- In the worst case scenario we have to look at every item in the array. That means it would run in O(n) complexity.

Linear or Sequential Search



Linear or Sequential Search

Eg: arr = [2, 12, 15, 11, 7, 19, 45]

Target element to search is 7.

- **Approach for Linear or Sequential Search**

1. Start with index 0 and compare each element with the target
2. If the target is found to be equal to the element, return its index
3. If the target is not found, return -1

Time Complexity Analysis

1. **Best case**

   The Best Case occurs when the target element is the first element of the array.

2. **Average case**

   The Average Case: On average, the target element will be somewhere in the middle of the array. The number of comparisons

3. **Worst case**

The Worst Case occurs when the target element is the last element in the array or not in the array.

Linear or Sequential Search

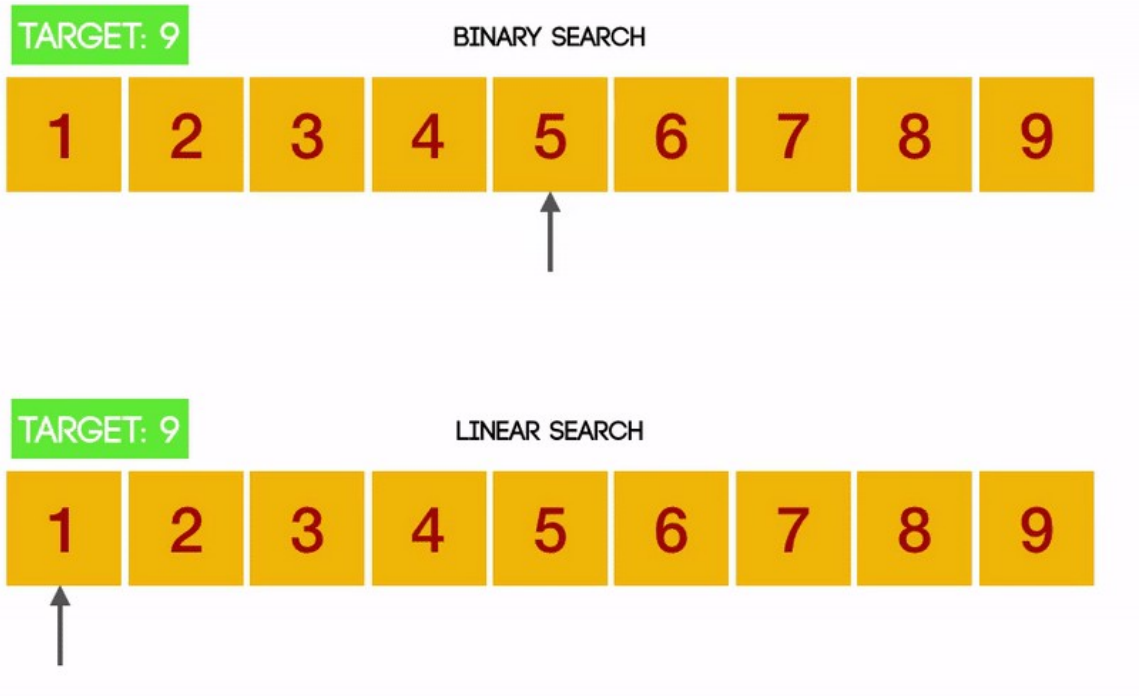
eg:

```
function linearSearch(array, toFind){
    for(let i = 0; i < array.length; i++){
      if(array[i] === toFind) return i;
    }
    return -1;
  }
  console.log(linearSearch([4,5,6],9));
```

Binary Search

- Binary search is more efficient than linear search.
- This type of searching algorithm is used to find the position of a specific value contained in a sorted array.
- The binary search algorithm works on the principle of divide and conquer and it is considered the best searching algorithm because it's faster to run.

Binary Search



Binary Search

Eg: arr = [2, 12, 15, 17, 27, 29, 45]

Suppose the target element to be searched is 17.

Approach for Binary Search

1. Compare the target element with the middle element of the array.
2. If the target element is greater than the middle element, then the search continues in the right half.
3. Else if the target element is less than the middle value, the search continues in the left half.
4. This process is repeated until the middle element is equal to the target element, or the target element is not in the array
5. If the target element is found, its index is returned, else -1 is returned.

Time complexity Analysis

1. **Best case**

   The Best Case occurs when the target element is the middle element of the array.

2. **Average case**

   The Average Case: On average, the target element will be somewhere in the array.

3. **Worst case**

   The Worst Case occurs when the target element is not in the list or it is away from the middle element.

Binary Search

eg:

```
function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length - 1;
  while (left <= right) {
    const mid = Math.floor((right + left) / 2);
    if (arr[mid] === target) {
      return mid;
    }
    if (arr[mid] < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }
  return -1;
}
```

Tasks

Task

1 Print all the duplicate elements in an array

arr=[10,6,7,8,10,5,7,3,5]

2 Find common elements in both arrays

arr1=[5,7,8,3,];

arr2=[7,8,2,1];